

Challenges in Machine Learning

Introduction

Machine Learning (ML) is not just about writing a few lines of code; it involves several real-world hurdles. This session moves from the theoretical "What is ML" to the practical "What problems will you face" in your career as a Data Scientist.

1. Data Collection

Machine Learning is entirely dependent on data. If there is no data, there is no Machine Learning.

Explanation:

- When you are a student or working on college projects, you get data easily in formats like .csv or .tsv from websites like Kaggle.
- In professional settings (companies), getting ready-made data is rare.
- You often have to build your own data collection systems.

Methods of Data Collection:

- **APIs:** Connecting to other software to get data.
- **Web Scraping:** Writing code to pull data from websites.

Key Challenges in Data Collection:

- Using code to bring data from a remote server to your own can lead to technical errors, server blocks, or inconsistent data formats.
- **Highlight:** If you don't have data, you can't even start the project.

2. Insufficient / Unlabelled Data

Even if you have data, is it enough? And is it tagged correctly?

The "Algorithm vs. Data" Concept:

The teacher explains a very important study regarding the "Unreasonable Effectiveness of Data."

Comparison Scenario:

- **Algorithm A:** A very complex, high-end algorithm (e.g., Deep Learning).
- **Algorithm B:** A very simple, basic algorithm.
- **Data 1:** 10,000 rows.
- **Data 2:** 1,000,000 rows.

Logic Explanation:

1. If Algorithm A (complex) works on 10k rows, its performance might be decent.
2. If Algorithm B (simple) works on 1M rows, it will likely **outperform** Algorithm A.
3. **Conclusion:** Having more data is often better than having a more complex algorithm.

The Labeling Problem:

- Collecting images is easy (e.g., downloading 10,000 images from Google).
- However, someone has to manually sit and "Label" them (e.g., "This is a cat," "This is a dog").
- This is a very tedious and time-consuming task.

3. Non-Representative Data

Your data must represent the real world. If it only represents a small part, your model will be "biased."

Explanation with Examples:

- **Case 1: Linear Regression Logic**
 - If you have a small amount of data, you might draw a straight line through it.
 - If you had the *full* dataset, you might see the data actually follows a curve.
 - Because your small sample didn't represent the curve, your model is wrong.
- **Case 2: The World Cup Survey**
 - **Goal:** Predict who will win the T20 World Cup.
 - **Action:** You only survey people in India.
 - **Result:** Most will say "India" because of emotional bias.
 - **The Error:** This data is not representative of the global cricket-watching population.

Two Types of Errors here:

1. **Sampling Noise:** When the data sample is too small, leading to random incorrect patterns.
2. **Sampling Bias:** When the method of collecting data is flawed (e.g., asking only Indians about a global match).

4. Poor Quality Data

"Garbage In, Garbage Out" (GIGO).

Explanation:

- Data is often "dirty." It contains:
 - **Noise:** Random errors.
 - **Missing Values:** Empty fields in the table.
 - **Outliers:** Values that are extremely different from the rest.
 - **Inconsistent Formats:** Dates written in different styles (DD/MM/YY vs MM/DD/YY).

Important Note for Students:

- **Highlight:** As a Data Scientist, you will spend **60% to 80%** of your time just cleaning and

- preparing data.
- Only 20% of your time goes into actual "Machine Learning" or modeling.

5. Irrelevant Features

Features are the columns (inputs) you give to your model.

Explanation:

- If you give useless information to your model, the results will be poor.
- **Example: Marathon Participation Prediction**
 - Inputs: Age, Weight, Height, Location.
 - Result: Will they run the marathon?
 - **Analysis:** Age, Weight, and Height are relevant to fitness. "Location" is likely irrelevant. If you keep the "Location" column, it adds "noise" and confuses the model.

Tip - Feature Engineering:

- This is the process of creating new, better features from old ones.
- **Example:** Instead of giving "Weight" and "Height" separately, you combine them to create "**BMI**" (Body Mass Index). One column (BMI) might tell a better story than two separate ones.

6. Overfitting

The model "over-learns" the training data.

Explanation:

- The model memorizes the data instead of understanding the underlying pattern.
- It performs perfectly on training data but fails on new, unseen data.

The "Expensive City" Example (Human Overfitting):

- You go to Gurgaon, visit one movie theater, and find the ticket costs ₹500.
- You conclude: "Everything in Gurgaon is expensive."
- **The Error:** You generalized too much based on one single point.

Technical Logic:

- If your model's curve tries to touch **every single point** in the training data, it is overfitting.
- It becomes too complex and loses the ability to "Generalize."

7. Underfitting

The opposite of Overfitting.

Explanation:

- The model is too simple to learn the patterns in the data.

- **Example:** If the data is complex and curvy, but you try to fit a simple straight line through it, you are underfitting.
- **Result:** The model performs poorly on both training data and new data.

8. Software Integration

ML models are useless if they don't live inside a software product.

Explanation:

- An ML model must eventually be part of a website, an Android app, or a system in an autonomous car.
- **The Problem:** Software runs on many platforms (Windows, Linux, Android, iOS).
- ML tools (mostly Python-based) are not always easily compatible with other languages like Java or JavaScript.
- Although tools like TensorFlow.js exist, they are still evolving and not as stable as traditional software languages.

9. Offline Learning and Deployment

How do you update your model?

Explanation:

- **Offline (Batch) Learning:** You train a model, put it on a server, and it stays the same. To update it with new data, you must take it "offline," retrain it, and upload it again.
- **Online Learning:** The model learns continuously on the server. This is very difficult to implement and monitor.
- **Deployment:** Putting a model on the cloud (AWS/Azure/GCP) is complex. It is not as "plug-and-play" as standard software yet.

10. Cost Involved

Machine Learning is expensive.

Explanation:

- **Hardware Cost:** Training models requires powerful GPUs and high RAM.
- **Cloud Cost:** Running a model for 100,000 users on the cloud can generate massive bills.
- **Hidden Costs:** Mentioned a paper called *"The Cost of AI"*. Many companies stop ML projects because the server costs are higher than the profit generated.

Final Summary & Career Tip

The teacher introduces a growing field called **MLOps** (Machine Learning Operations).

- **MLOps:** Combining Machine Learning + DevOps.
- It focuses on how to handle the "Operations" (deployment, cost, scaling, monitoring) of ML

models.

- **Student Advice:** Don't just learn to make a model; learn how to convert that model into a **product** that a user can actually use on a website or app.

Key Highlight: The real challenge isn't just the math; it's the data, the integration, and the cost.